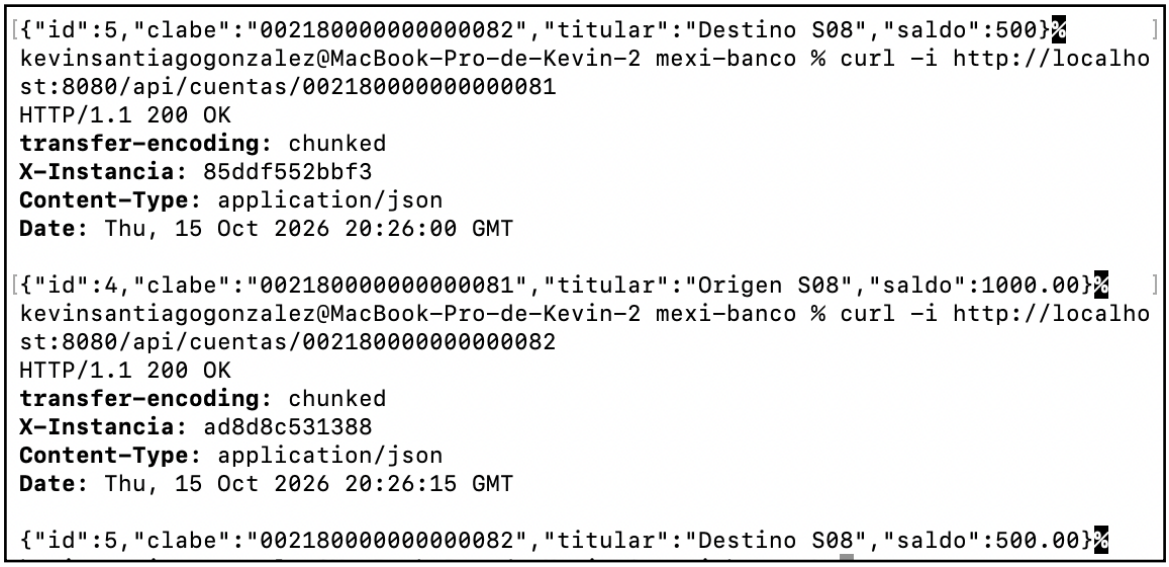
Lab S08. Saga y circuit breaker

Kevin Santiago González | Grupo 2
Sistemas Distribuidos 2027-1
15 de octubre de 2026

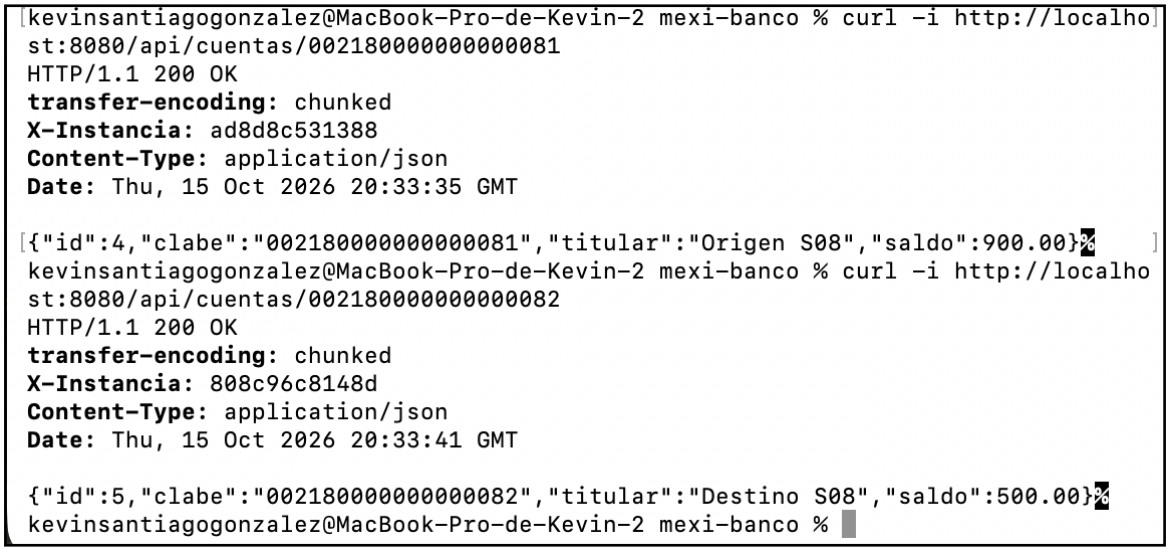
1. El problema

Primero probé la versión original. Puse una pausa de 20 segundos entre el cargo y el abono y apagué cuenta durante esa pausa. Al volver a encenderla, el origen había perdido 100 y el destino seguía igual.

Cuenta	Antes	Después
Origen: 002180000000000081	1000.00	900.00
Destino: 002180000000000082	500.00	500.00



Saldos antes de la prueba.

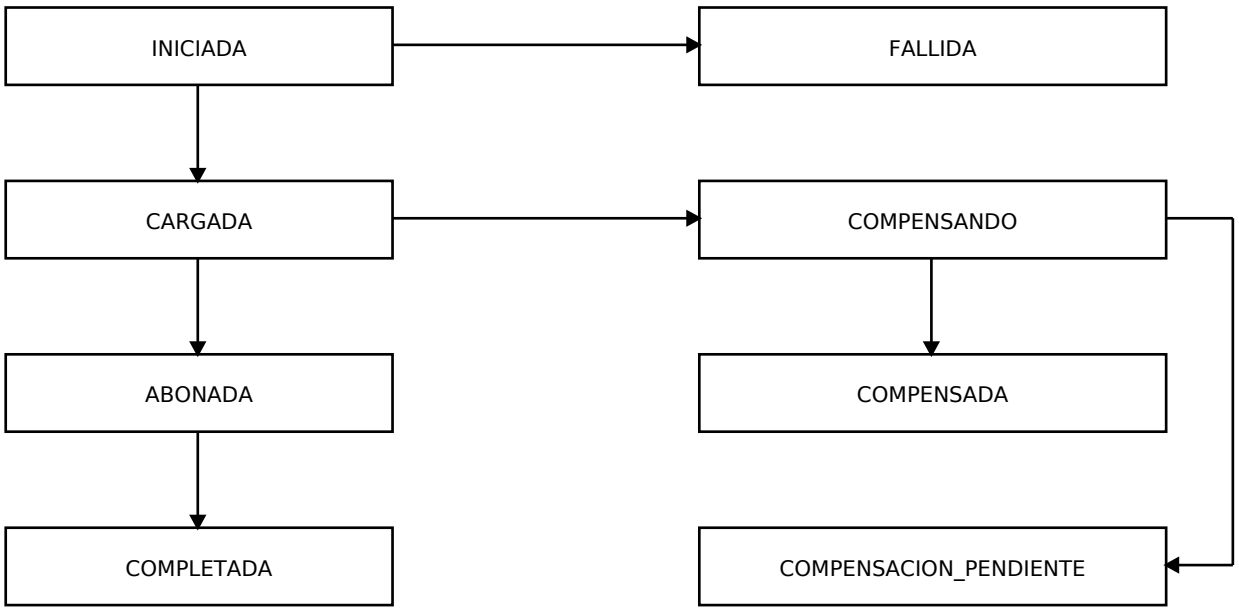


Saldos después de apagar y volver a encender cuenta.

Pasó porque cada llamada HTTP se confirma por separado. El cargo ya estaba guardado cuando falló el abono. El cliente recibió un 504, pero eso no deshizo el cargo.

2. Mi saga

TransferenciaService es el orquestador. Crea el registro antes de mover dinero y guarda el estado después de cada paso. Si el abono no se aplicó, intenta devolver el monto al origen.



Ruta izquierda: cargo, abono y registro de movimientos. A la derecha: rechazo antes del cargo, devolución confirmada o devolución pendiente. Si no se sabe si una escritura se aplicó, también se guarda COMPENSACION_PENDIENTE para revisarla.

Paso	**Qué hago si falla**
Consulta de cuentas	Marco FALLIDA; todavía no moví dinero.
Abono no aplicado, cargo confirmado	Devuelvo el monto al origen y registro el cargo y la devolución.
Devolución	Dejo COMPENSACION_PENDIENTE si no puedo confirmarla.
Movimientos o avisos después del abono	Reviso lo que falta sin devolver dinero otra vez.

La tabla guarda el último estado y los logs muestran los cambios. No agregué recuperación automática si transferencia se cae entre una llamada remota y el guardado del estado.

3. Código que cambié

Estado de la transferencia

```
@Enumerated(EnumType.STRING)
@Column(length = 40)
private Estado estado;

@Column(length = 500)
private String detalle;
```

Agregué estado y detalle a Transferencia.java. Las nuevas operaciones empiezan en INICIADA. El detalle explica al cliente qué pasó.

```
private void guardarEstado(Transferencia transferencia,
        Transferencia.Estado estado, String detalle) {
    transferencia.actualizarEstado(estado, detalle);
    transferencias.save(transferencia);
    log.info("Saga {}: {} - {}",
        transferencia.getId(), estado, detalle);
}
```

Uso este método para guardar cada avance. transferir() no tiene una transacción que abarque toda la operación; los guardados locales se confirman por separado.

Falla de prueba y devolución

```
@Value("${FALLAR_ABONO_CLABE:}")
private String fallarAbonoClabe = "";

// Después del cargo, antes de enviar el abono:
if (claveDestino.equals(fallarAbonoClabe)) {
    return compensar(transferencia, origen);
}
```

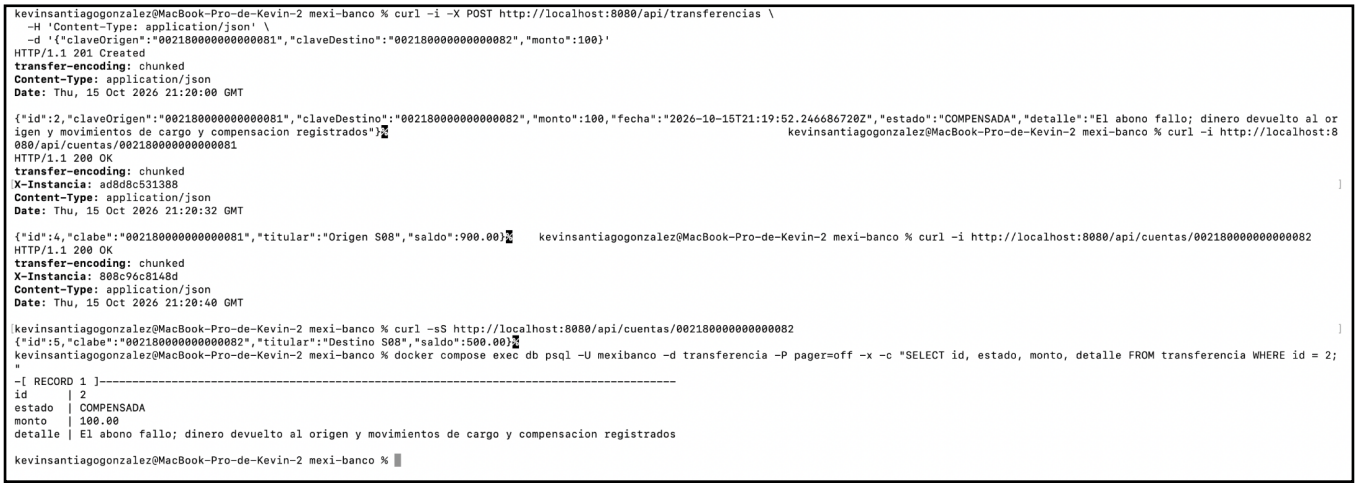
Fragmento resumido. Usé FALLAR_ABONO_CLABE=002180000000000082 y pausa en 0. Así falla el abono al destino sin apagar cuenta ni bloquear la devolución al origen.

```
origenRestaurado = cuentas.abonar(
    transferencia.getClaveOrigen(),
    transferencia.getMonto()
);
```

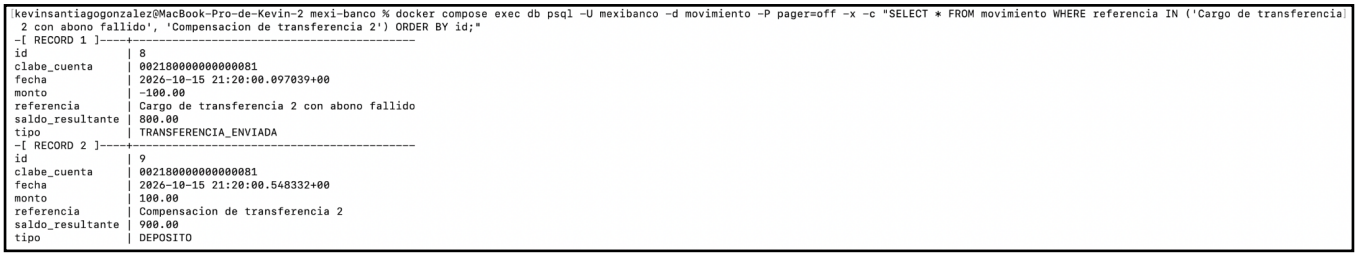
Antes de devolver guardo COMPENSANDO. Si devuelve bien, guardo COMPENSADA y registro los movimientos. Si falla, queda pendiente. Separé la devolución del registro de movimientos para no devolver dos veces si falla ese registro. Un timeout puede tener resultado incierto, así que no reintento la escritura a ciegas.

4. La compensación funcionando

En la transferencia 2 envié 100 con la falla simulada. Empecé con 900 en origen y 500 en destino. Al terminar quedaron otra vez 900 y 500, y la base guardó COMPENSADA.



Respuesta, saldos y estado de la transferencia 2.



Movimientos del cargo y la devolución.

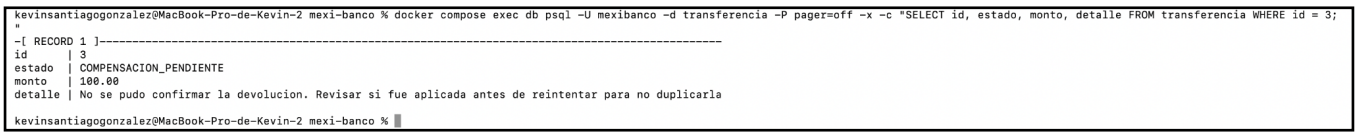
Movimiento	Monto	Saldo de origen
Cargo (ID 8)	-100.00	800.00
Devolución (ID 9)	+100.00	900.00

La devolución quedó como DEPOSITO, con la referencia “Compensacion de transferencia 2”. El origen regresó a 900, no a 1000, porque los primeros 100 se perdieron en la prueba de la versión original.

Esta prueba de compensación se hizo antes de integrar el circuit breaker. Después probé la apertura y recuperación del circuito.

5. Cuando también falla la compensación

Quité la falla simulada, puse la pausa en 20 segundos y apagué cuenta después del cargo. La transferencia 3, de 100, quedó en COMPENSACION_PENDIENTE.



Estado guardado cuando no se pudo confirmar la devolución.

Al encender cuenta, el origen tenía 800 y el destino 500. La devolución sigue pendiente; reiniciar el servicio no la realiza automáticamente.

Cómo la terminaría

Agregaría un proceso que revise operaciones pendientes. Primero comprobaría si el cargo, el abono o la devolución se aplicaron. Después devolvería lo que corresponda usando una clave fija, como compensacion:3.

La compensación debe ser idempotente: si cuenta devuelve el dinero pero se pierde la respuesta, repetir el intento no debe devolverlo otra vez. Para eso, cuenta tendría que guardar la clave y el cambio de saldo en la misma transacción y reconocer los intentos repetidos. Esto quedó como propuesta, no está implementado.

6. Circuit breaker a mano

Estado	Qué hace
CERRADO	Deja pasar llamadas. Abre tras 3 fallas seguidas de disponibilidad.
ABIERTO	Rechaza sin llamar a cuenta. Espera 30 segundos.
SEMIABIERTO	Deja pasar una prueba. Si responde, cierra; si falla por disponibilidad, abre otra vez.

Creé CircuitoCuenta y lo usé en consultar(), cargar() y abonar(). Conservé los timeouts existentes: 2 segundos para conectar y 5 de lectura. El circuito también protege la devolución.

6. Prueba del circuito y aislamiento

Con cuenta apagada envié cinco transferencias de 10. Las tres primeras fallas abrieron el circuito; las dos siguientes se rechazaron sin llamar a cuenta. Todas quedaron FALLIDA antes del cargo.

```
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco % for intento in 1 2 3 4 5
do
  printf '\nIntento %s\n' "$intento"
  curl -sS \
    -w '\nHTTP: %{http_code} | Tiempo: %{time_total} segundos\n' \
    -X POST http://localhost:8080/api/transferencias \
    -H 'Content-Type: application/json' \
    -d '{"claveOrigen":"00218000000000081","claveDestino":"00218000000000082","monto":10}'
done

Intento 1
{"id":4,"claveOrigen":"00218000000000081","claveDestino":"00218000000000082","monto":10,"fecha":"2026-10-15T21:42:47.168957384Z","estado":"FALLIDA","detalle":"No fue posible validar las cuentas. No se
 intento ningun cargo"}
HTTP: 201 | Tiempo: 1.032987 segundos

Intento 2
{"id":5,"claveOrigen":"00218000000000081","claveDestino":"00218000000000082","monto":10,"fecha":"2026-10-15T21:42:47.721052134Z","estado":"FALLIDA","detalle":"No fue posible validar las cuentas. No se
 intento ningun cargo"}
HTTP: 201 | Tiempo: 0.039569 segundos

Intento 3
{"id":6,"claveOrigen":"00218000000000081","claveDestino":"00218000000000082","monto":10,"fecha":"2026-10-15T21:42:47.764848884Z","estado":"FALLIDA","detalle":"No fue posible validar las cuentas. No se
 intento ningun cargo"}
HTTP: 201 | Tiempo: 0.024488 segundos

Intento 4
{"id":7,"claveOrigen":"00218000000000081","claveDestino":"00218000000000082","monto":10,"fecha":"2026-10-15T21:42:47.803431301Z","estado":"FALLIDA","detalle":"No fue posible validar las cuentas. No se
 intento ningun cargo"}
HTTP: 201 | Tiempo: 0.021707 segundos

Intento 5
{"id":8,"claveOrigen":"00218000000000081","claveDestino":"00218000000000082","monto":10,"fecha":"2026-10-15T21:42:47.836572759Z","estado":"FALLIDA","detalle":"No fue posible validar las cuentas. No se
 intento ningun cargo"}
HTTP: 201 | Tiempo: 0.022527 segundos
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco % docker compose logs --no-color transferencia | awk '/CB cuenta:/'
transferencia-1  | 2026-10-15T21:41:22.006Z  INFO 1 --- [transferencia] [           main] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: estado inicial CERRADO
transferencia-1  | 2026-10-15T21:42:47.371Z  INFO 1 --- [transferencia] [nio-8080-exec-7] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: llamada autorizada: consultar
transferencia-1  | 2026-10-15T21:42:47.586Z  WARN 1 --- [transferencia] [nio-8080-exec-7] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: CERRADO; falla 1/3 en consultar
transferencia-1  | 2026-10-15T21:42:47.728Z  INFO 1 --- [transferencia] [nio-8080-exec-9] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: llamada autorizada: consultar
transferencia-1  | 2026-10-15T21:42:47.731Z  WARN 1 --- [transferencia] [nio-8080-exec-9] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: CERRADO; falla 2/3 en consultar
transferencia-1  | 2026-10-15T21:42:47.767Z  INFO 1 --- [transferencia] [io-8080-exec-10] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: llamada autorizada: consultar
transferencia-1  | 2026-10-15T21:42:47.769Z  WARN 1 --- [transferencia] [io-8080-exec-10] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: CERRADO; falla 3/3 en consultar
transferencia-1  | 2026-10-15T21:42:47.769Z  WARN 1 --- [transferencia] [io-8080-exec-10] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: CERRADO -> ABIERTO; se alcanzo el limite de fallas; espera
 de 30 segundos
transferencia-1  | 2026-10-15T21:42:47.806Z  WARN 1 --- [transferencia] [nio-8080-exec-1] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: ABIERTO; rechazo consultar sin llamar a cuenta
transferencia-1  | 2026-10-15T21:42:47.841Z  WARN 1 --- [transferencia] [nio-8080-exec-2] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: ABIERTO; rechazo consultar sin llamar a cuenta
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco %
```

CERRADO, tres fallas y ABIERTO. Los últimos intentos tardaron unos 0.02 segundos.

Encendí cuenta y esperé su registro y el tiempo de apertura. La siguiente solicitud activó SEMIABIERTO y, al responder cuenta, pasó a CERRADO.

```
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco % docker compose logs --no-color transferencia | awk '/CB cuenta:/'
transferencia-1  | 2026-10-15T21:41:22.006Z  INFO 1 --- [transferencia] [           main] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: estado inicial CERRADO
transferencia-1  | 2026-10-15T21:42:47.371Z  INFO 1 --- [transferencia] [nio-8080-exec-7] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: llamada autorizada: consultar
transferencia-1  | 2026-10-15T21:42:47.586Z  WARN 1 --- [transferencia] [nio-8080-exec-7] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: CERRADO; falla 1/3 en consultar
transferencia-1  | 2026-10-15T21:42:47.728Z  INFO 1 --- [transferencia] [nio-8080-exec-9] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: llamada autorizada: consultar
transferencia-1  | 2026-10-15T21:42:47.731Z  WARN 1 --- [transferencia] [nio-8080-exec-9] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: CERRADO; falla 2/3 en consultar
transferencia-1  | 2026-10-15T21:42:47.767Z  INFO 1 --- [transferencia] [io-8080-exec-10] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: llamada autorizada: consultar
transferencia-1  | 2026-10-15T21:42:47.769Z  WARN 1 --- [transferencia] [io-8080-exec-10] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: CERRADO; falla 3/3 en consultar
transferencia-1  | 2026-10-15T21:42:47.769Z  WARN 1 --- [transferencia] [io-8080-exec-10] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: CERRADO -> ABIERTO; se alcanzo el limite de fallas; espera
 de 30 segundos
transferencia-1  | 2026-10-15T21:42:47.806Z  WARN 1 --- [transferencia] [nio-8080-exec-1] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: ABIERTO; rechazo consultar sin llamar a cuenta
transferencia-1  | 2026-10-15T21:42:47.841Z  WARN 1 --- [transferencia] [nio-8080-exec-2] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: ABIERTO; rechazo consultar sin llamar a cuenta
transferencia-1  | 2026-10-15T21:45:18.673Z  INFO 1 --- [transferencia] [nio-8080-exec-3] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: ABIERTO -> SEMIABIERTO; se permite una llamada de prueba
transferencia-1  | 2026-10-15T21:45:18.673Z  INFO 1 --- [transferencia] [nio-8080-exec-3] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: llamada autorizada: consultar
transferencia-1  | 2026-10-15T21:45:19.563Z  INFO 1 --- [transferencia] [nio-8080-exec-3] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: SEMIABIERTO -> CERRADO; cuenta respondio
transferencia-1  | 2026-10-15T21:45:19.563Z  INFO 1 --- [transferencia] [nio-8080-exec-3] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: llamada autorizada: consultar
transferencia-1  | 2026-10-15T21:45:19.922Z  INFO 1 --- [transferencia] [nio-8080-exec-3] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: llamada autorizada: cargar
transferencia-1  | 2026-10-15T21:45:20.435Z  INFO 1 --- [transferencia] [nio-8080-exec-3] mx.mexibanco.clientes.CircuitoCuenta     : CB cuenta: llamada autorizada: abonar
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco %
```

Logs de ABIERTO a SEMIABIERTO y después a CERRADO.

```
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco % curl -i -X POST http://localhost:8080/api/transferencias \
  -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"00218000000000081","claveDestino":"00218000000000082","monto":10}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Thu, 15 Oct 2026 21:45:23 GMT

{"id":9,"claveOrigen":"00218000000000081","claveDestino":"00218000000000082","monto":10,"fecha":"2026-10-15T21:45:18.556822718Z","estado":"COMPLETADA","detalle":"Transferencia completada, movimientos
registrados y avisos enviados"}%
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco % curl -sS http://localhost:8080/api/cuentas/00218000000000081
{"id":4,"clabe":"00218000000000081","titular":"Origen S08","saldo":790.00}%
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco % curl -sS http://localhost:8080/api/cuentas/00218000000000082
{"id":5,"clabe":"00218000000000082","titular":"Destino S08","saldo":510.00}%
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco %
```

Transferencia 9 COMPLETADA: origen 790.00 y destino 510.00.

7. Lo que la saga no me da

La saga no aísla todos los pasos. Entre el cargo y la devolución, otra persona puede consultar el origen y ver 800 en lugar de 900. Incluso podría fallar otro pago por ese saldo temporal. La compensación devuelve el dinero, pero no borra lo que otros vieron o decidieron mientras tanto.

Hice las pruebas manuales mostradas. La construcción Docker usó -DskipTests; no ejecuté la suite automática. No implementé reintentos automáticos ni idempotencia en cuenta.